Week 7: Linear Models

CSCI 447: Machine Learning– Fall 2026

Bayesian Classification

- In classification, we want to maximize $P(f_h|\mathcal{D})$.
- Using the results from Bayesian decision theory, this means we want

$$class = \text{argmax}_{c \in \mathcal{C}} P(c|f_1, \dots, f_d).$$

- As we did with Bayes decision theory, we will apply Bayes rule.

$$P(c|f_1, \dots f_d) = \frac{P(f_1, \dots f_d|c)P(c)}{P(f_1, \dots, f_d)}.$$

- This looks like it's expensive!

Complexity

- Assume the random variables $c, f_1, \ldots, f_d$ are all Boolean.
- There are a total of $d + 1$ random variables.
- Consider the distribution, $P(f_1, \ldots, f_d | c)$.
- Fully parameterizing will require 2^{d+1} probabilities.
- Problems:
  - Storage
  - Computational time
  - Excessively large number of parameters to estimate

Conditional Independence

- **Def:** Two random variables X, Y are conditionally independent given a third random variable Z, denoted $(X \perp Y|Z)$, if any of the following hold:

$$P(X, Y|Z) = P(X|Z)P(Y|Z)$$

$$P(X|Y, Z) = P(X|Z)$$

$$P(Y|X, Z) = P(Y|Z)$$

- All of these are equivalent.
- For our classifier, assume $\forall i, j, i \neq j, (f_i \perp f_j|c)$. Then

$$P(c|f_1, \dots f_d) = \frac{P(f_1|c) \times \cdots \times P(f_d|c)P(c)}{P(f_1, \dots, f_d)}$$

$$class = \text{argmax}_{c \in \mathcal{C}}\, P(c) \prod_{i=1}^{d} P(f_i|c)\,.$$

Example

Class	F1	F2	F3	F4	F5
0	1	1	1	0	0
0	0	1	1	1	0
0	0	0	1	1	1
0	1	0	0	1	0
0	0	1	1	0	1
1	0	1	0	1	0
1	1	0	0	0	0
1	0	1	0	0	0
1	1	0	1	0	1
1	0	1	0	1	0

Estimate $P(c)$:

$$P(c = 0) = \frac{5}{10} = 0.5$$

$$P(c = 1) = \frac{5}{10} = 0.5$$

Estimate $P(F1 = 0|c = 0)$:

$$P(F1 = 0|c = 0) = \frac{3}{5} = 0.6$$

Example

$P(C=0)$ = 0.5	$P(F1=0 \mid C=0)$ = 0.6	$P(F2=0 \mid C=0)$ = 0.4	$P(F3=0 \mid C=0)$ = 0.2	$P(F4=0 \mid C=0)$ = 0.4	$P(F5=0 \mid C=0)$ = 0.6
	$P(F1=1 \mid C=0)$ = 0.4	$P(F2=1 \mid C=0)$ = 0.6	$P(F3=1 \mid C=0)$ = 0.8	$P(F4=1 \mid C=0)$ = 0.6	$P(F5=1 \mid C=0)$ = 0.4
$P(C=1)$ = 0.5	$P(F1=0 \mid C=1)$ = 0.6	$P(F2=0 \mid C=1)$ = 0.4	$P(F3=0 \mid C=1)$ = 0.8	$P(F4=0 \mid C=1)$ = 0.6	$P(F5=0 \mid C=1)$ = 0.8
	$P(F1=1 \mid C=1)$ = 0.4	$P(F2=1 \mid C=1)$ = 0.6	$P(F3=1 \mid C=1)$ = 0.2	$P(F4=1 \mid C=1)$ = 0.4	$P(F5=1 \mid C=1)$ = 0.2

Classify [1 1 0 1 0]

$$C = 0: 0.5 \times 0.4 \times 0.6 \times 0.2 \times 0.6 \times 0.6 = 0.00864$$

$$C = 1: 0.5 \times 0.4 \times 0.6 \times 0.8 \times 0.4 \times 0.8 = \mathbf{0.03072}$$

Decide $C = 1$.

Subtleties

- Naïve Bayes can only solve *linearly separable* problems exactly.
- A problem arises if any of the conditional probabilities equals zero.
  $\Longrightarrow$ Smooth
- m-estimate:

$$P(f|c) \approx \frac{n_{f,c} + mp}{n_c + m}$$

- Example: Suppose $m = 1$ and $p = 0.001$.

$$\frac{0 + 0.001}{n_c + 1} \neq 0$$

Fundamental of Gaussian Naïve Bayes

- Focused on working with continuous (real-valued) features.
- Still assumes discrete classes (i.e., not used for regression).
- Prior class probability:

$$P(class)$$

- Variable distribution is Gaussian:

$$X \sim \mathcal{N}(\mu, \sigma^2) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

- Assume d random variables: $X_1, \ldots, X_d$ and class variable Y.
- Our goal is to estimate:

$$\mu_{ik} = E[X_i | Y = y_k]$$
$$\sigma_{ik}^2 = E[(X_i - \mu_{ik})^2 | Y = y_k]$$

Maximum Likelihood Estimation

- MLE estimate for the mean:

$$\hat{\mu}_{ik} = \frac{1}{\sum_j \delta(Y^j = y_k)} \sum_j X_i^j \delta(Y^j = y_k)$$

where

$$(Y^j = y_k) = \begin{cases} 1 & Y^j = y_k \\ 0 & \text{otherwise} \end{cases}$$

- MLE estimate for the variance:

$$\hat{\sigma}_{ik}^2 = \frac{1}{\sum_j \delta(Y^j = y_k)} \sum_j \left(X_i^j - \hat{\mu}_{ik}\right)^2 \delta(Y^j = y_k)$$

unbiased:

$$\hat{\sigma}_{ik}^2 = \frac{1}{(\sum_j \delta(Y^j = y_k)) - 1} \sum_j \left(X_i^j - \hat{\mu}_{ik}\right)^2 \delta(Y^j = y_k)$$

GNB Rule

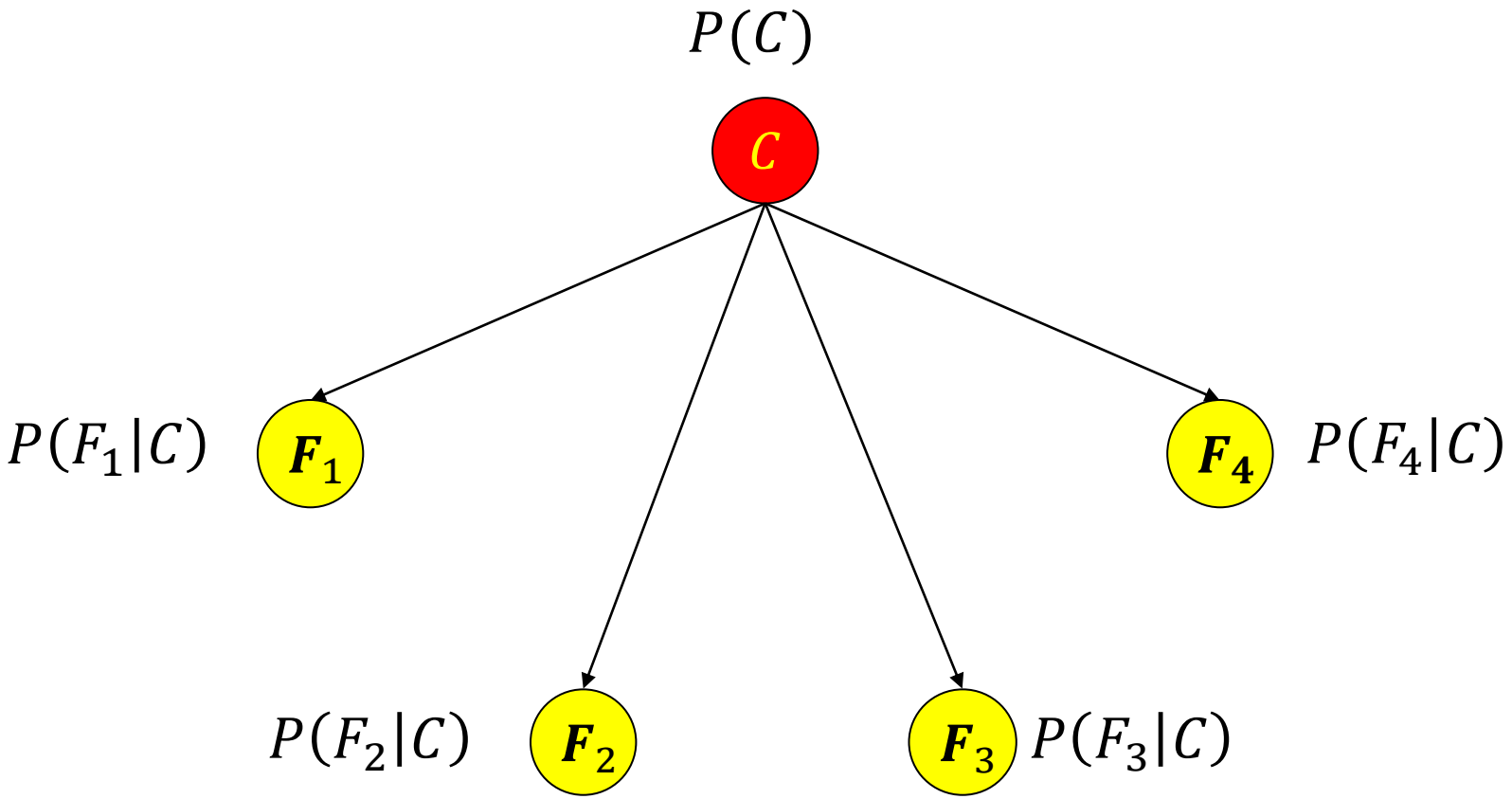
- The Gaussian Naïve Bayes rule then becomes

$$class = \underset{c \in C}{\operatorname{argmax}} P(c) \prod_{i=1}^{d} P(f_i|c)$$

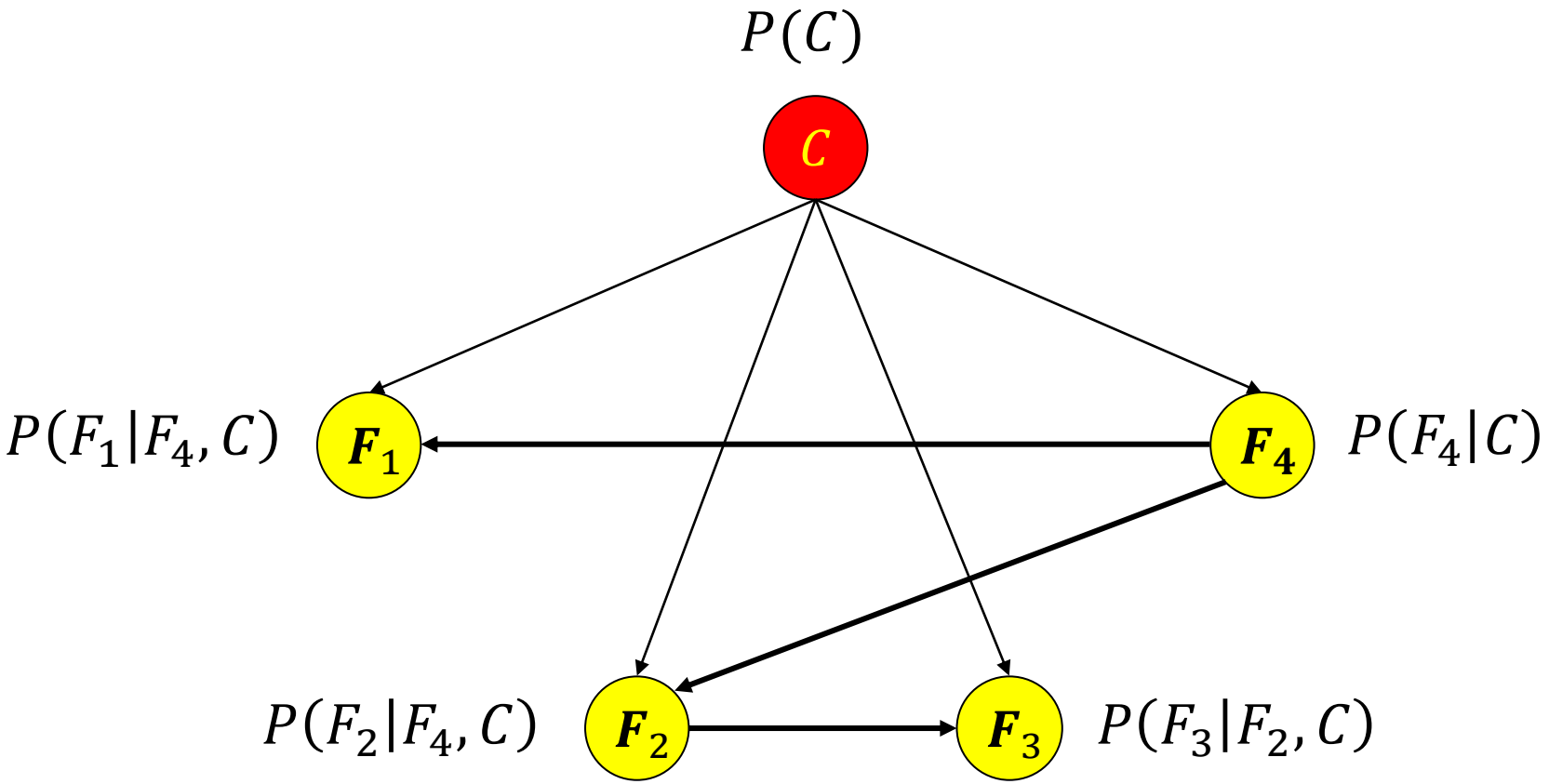
- Substituting in the Gaussians gives us

$$class = \underset{c \in C}{\operatorname{argmax}} P(c) \prod_{i=1}^{d} \mathcal{N}(\hat{\mu}_{ic}, \hat{\sigma}_{ic}) .$$

Tree Augmented Naïve Bayes



Tree Augmented Naïve Bayes



Classification Rule

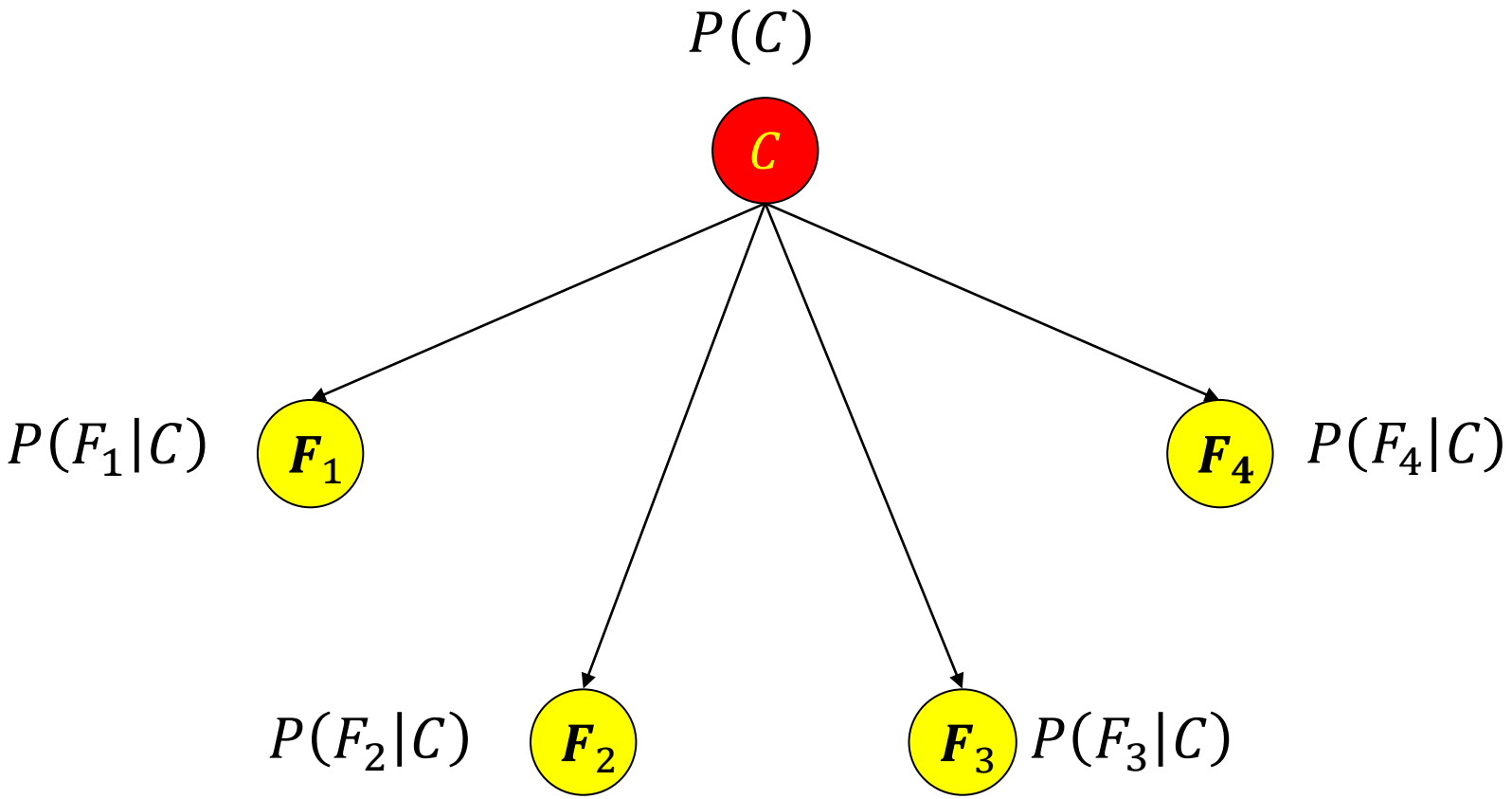
- To classify using the augmented structure.

$$class = \underset{c \in C}{\operatorname{argmax}} P(c) P(f_{\text{root}} | c) \prod_{(f_i, f_j) \in TAN} P(f_i | f_j, c)$$

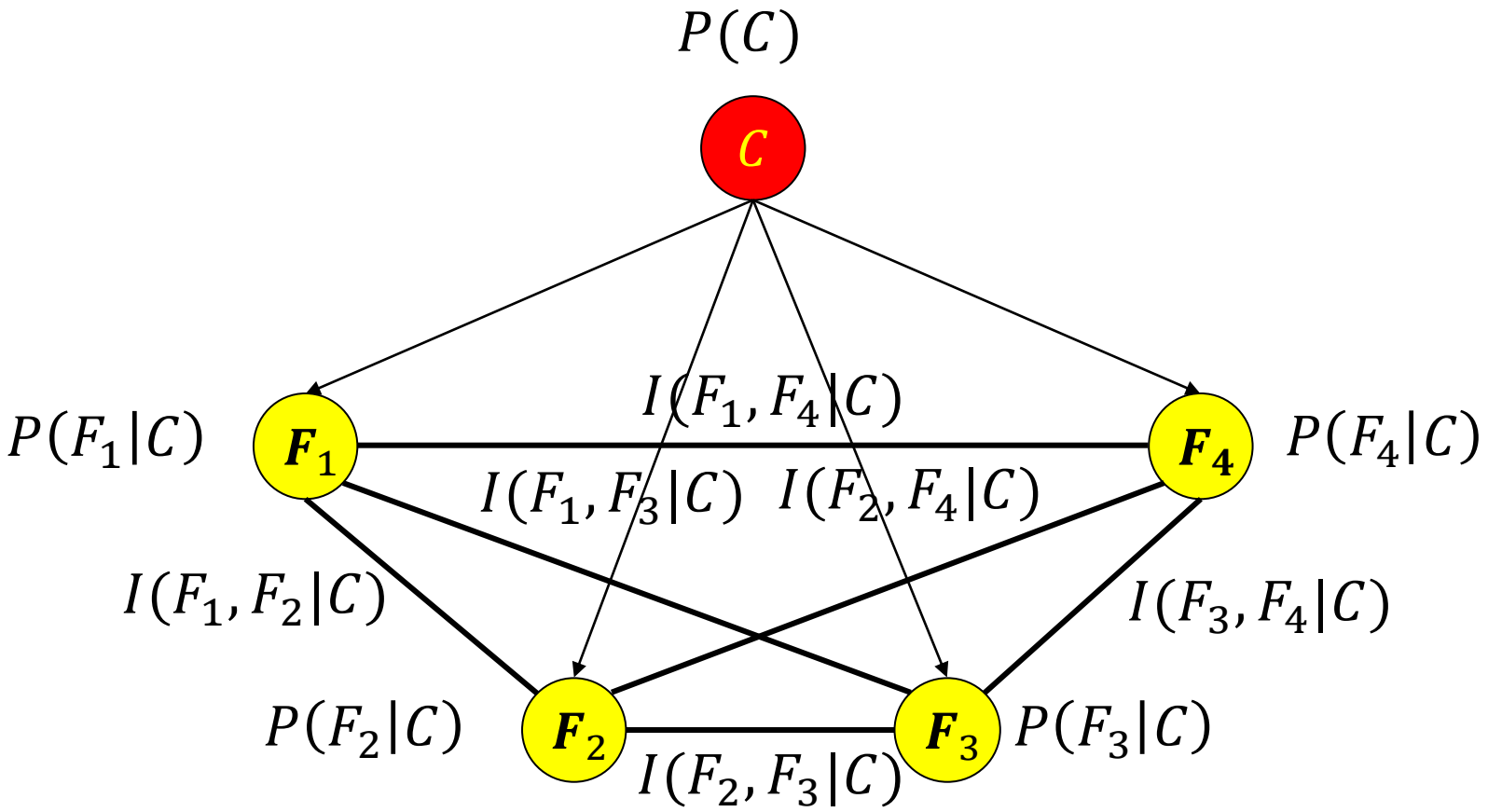
- Use conditional mutual information.

$$I(F_i, F_j | C) = \sum_{F_i, F_j, C} P(F_i, F_j, C) \log \frac{P(F_i, F_j | C)}{P(F_i | C) P(F_j | C)}$$

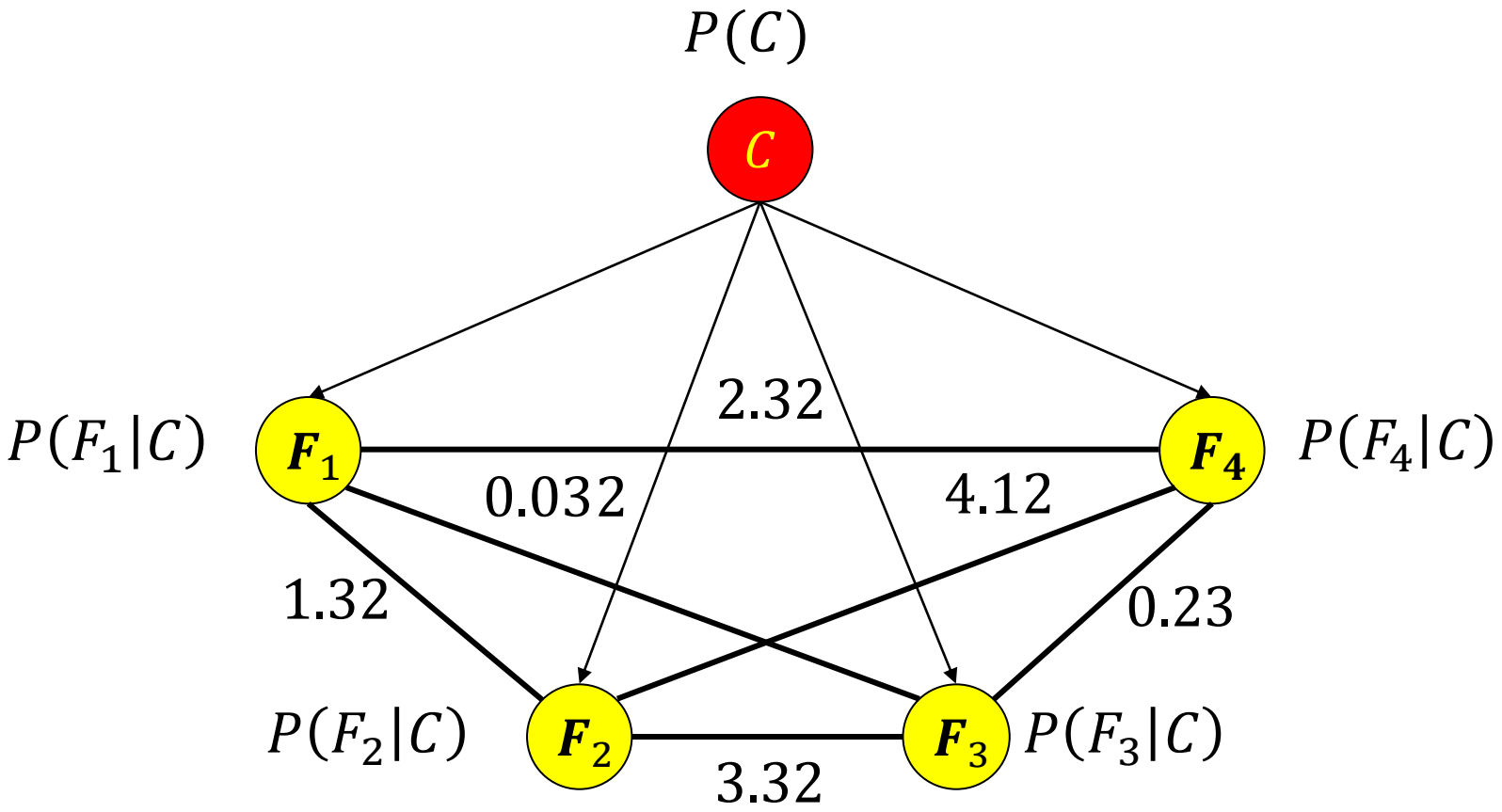
Building a TAN



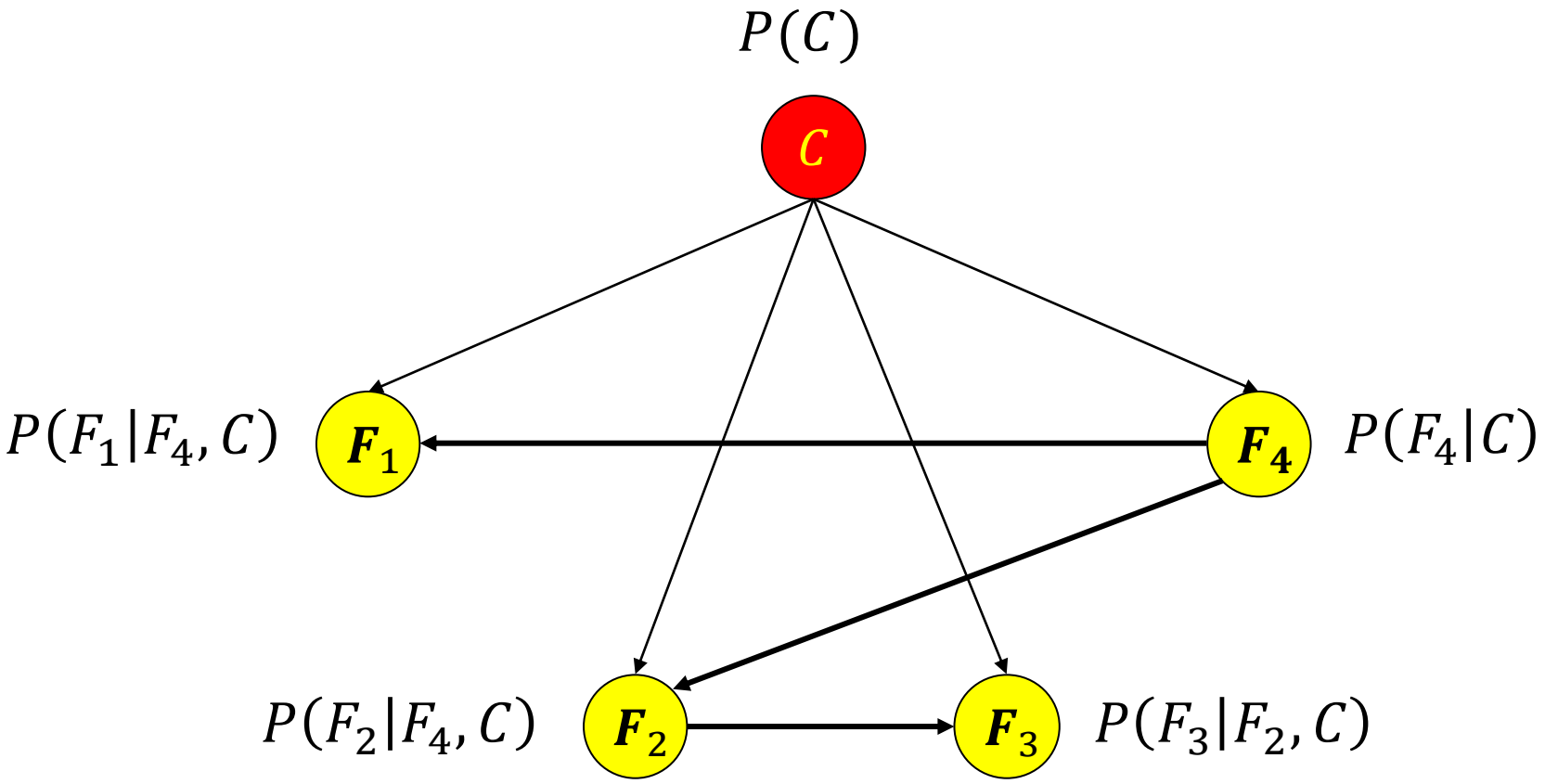
Building a TAN



Building a TAN



Building a TAN



Optimization

- To this point, we have been approaching machine learning as an optimization problem.
- Specifically, we seek to find model parameters $\boldsymbol{\Theta}$ such that some loss function $\mathcal{L}(f(\mathcal{D}, \boldsymbol{\Theta}))$ is minimized over data set $\mathcal{D}$.
- If we know the functional form of the model to be learned, we can apply analytical methods for optimization:

$$\boldsymbol{\Theta} = \underset{\boldsymbol{\theta} \in \boldsymbol{\Theta}^*}{\operatorname{argmin}} \mathcal{L}(f(\mathcal{D}, \boldsymbol{\theta}))$$

$$\nabla_{\boldsymbol{\Theta}} \mathcal{L}(f(\mathcal{D}, \boldsymbol{\Theta})) = 0$$

Solve for $\boldsymbol{\Theta}$.

Gradient Descent

- Generally, we do not know the complete functional form of the loss function due to its dependence on the model's functional form $f(\mathcal{D}, \boldsymbol{\Theta})$.
- Furthermore, the resulting function is likely to be highly nonlinear, resulting in
  - Several critical points (minima and saddle points)
  - Some critical points worse than others (local minima)
- Finding the global optimum is therefore intractable.
- Therefore, we approximate the solution by computing a local gradient of the loss $\mathcal{L}(f(\mathcal{D}, \boldsymbol{\Theta}))$ and following the gradient to the nearest local optimum.

One-Step Gradient Descent

- Select an initial point in the search space: $\boldsymbol{\theta}^0$
- Repeat
  - $t \leftarrow t + 1$
  - Calculate the gradient of the loss function at that point: $\nabla_{\boldsymbol{\theta}} \mathcal{L}(f(\mathcal{D}, \boldsymbol{\theta}))$
  - Update the parameters: $\theta^t \leftarrow \theta^{t-1} - \eta \nabla_{\boldsymbol{\theta}} \mathcal{L}(f(\mathcal{D}, \boldsymbol{\theta}^{t-1})$
- Until some convergence property is met.

- Here $\eta \in (0,1)$ is a learning rate, corresponding to the "step size" of the update.

Types of Gradient Descent

- Generally, three types of gradient descent are employed:
  - Batch gradient descent
  - Incremental (stochastic) gradient descent
  - Mini-batch (stochastic) gradient descent
- Technically, the incremental approach is a type of mini-batch where the batch size is set equal to one.
- Key to mini-batch gradient descent is randomization; this is the stochastic part.

Batch Gradient Descent

- Assume we have a training set, $\mathcal{D} = \{\mathbf{x}^i, y^i\}_{i=1}^{|\mathcal{D}|}$.
- Initialize parameters $\boldsymbol{\theta}^0$
- Repeat
  - $t \leftarrow t + 1$
  - $\Delta\boldsymbol{\theta} \leftarrow 0$
  - For $i = 1, |\mathcal{D}|$ do
    - $\Delta\boldsymbol{\theta} \leftarrow \Delta\boldsymbol{\theta} - \eta_t \nabla_\theta \mathcal{L}(f(x^i, \boldsymbol{\theta}^{t-1}))$
  - End For
  - $\boldsymbol{\theta}^t \leftarrow \boldsymbol{\theta}^{t-1} + \frac{1}{|\mathcal{D}|}\Delta\boldsymbol{\theta}$
- Until some convergence property is met

Incremental Gradient Descent

- Assume we have a training set, $\mathcal{D} = \{\mathbf{x}^i, y^i\}_{i=1}^{|\mathcal{D}|}$.
- Initialize parameters $\boldsymbol{\theta}^0$
- Repeat
  - Generate a random permutation of $\mathcal{D}$
  - For $i = 1, |\mathcal{D}|$ do
    - $t \leftarrow t + 1$
    - $\Delta\boldsymbol{\theta} \leftarrow -\eta_t \nabla_\theta \mathcal{L}(f(x^i, \boldsymbol{\theta}^{t-1}))$
    - $\boldsymbol{\theta}^t \leftarrow \boldsymbol{\theta}^{t-1} + \Delta\boldsymbol{\theta}$
  - End For
- Until some convergence property is met

Mini-batch Gradient Descent

- Assume we have a training set, $\mathcal{D} = \left\{\mathbf{x}^i, y^i\right\}_{i=1}^{|\mathcal{D}|}$.
- Initialize parameters $\boldsymbol{\theta}^0$
- Repeat
  - $t \leftarrow t + 1$
  - Let $\mathcal{B}^t \sim \mathbb{U}(\mathcal{D})$ be a randomly generated mini-batch of some size $|\mathcal{B}^t| < |\mathcal{D}|$
  - $\Delta\boldsymbol{\theta} \leftarrow 0$
  - For $i = 1, |\mathcal{B}^t|$ do
    - $\Delta\boldsymbol{\theta} \leftarrow \Delta\boldsymbol{\theta} - \eta_t \nabla_\theta \mathcal{L}(f(x^i, \boldsymbol{\theta}^{t-1}))$
  - End For
  - $\boldsymbol{\theta}^t \leftarrow \boldsymbol{\theta}^{t-1} + \frac{1}{|\mathcal{B}^t|}\delta\boldsymbol{\theta}$
- Until some convergence property is met

The Learning Rate

- Much of the behavior of gradient descent is driven by the selected learning rate.
- It is critical that the learning rate be tuned.
- In appropriate learning rates can have negative results:
  - Too small – slow convergence
  - Too large – unstable convergence due to oscillation (over-shooting)
- Ideally:
  - Want the learning rate to be large when we have long, shallow gradients
  - Want the learning rate to be small when we have short, steep gradients
- Gradient magnitudes are determined directly, but the length is difficult to determine.
- One approach to handling is to set the initial learning rate large and gradually reduce over time (referred to as “annealing”).
- Adaptive learning rate methods exist (e.g., Adam) but still need a starting point.

Convergence

- Gradient descent, in all of its forms, is an iterative search process.
- As such, we need to have a way to determine when to stop the search.
- There are a number of common approaches that can be considered:
  - Pre-set a maximum number of iterations and stop when that number is reached (difficult to manage over-fitting or under-fitting; should be tuned)
  - Keep track of a recent history of updates; when the magnitudes of the updates appear to stagnate, stop (can lead to getting trapped on a ridge or plateau, or can lead to overfitting)
  - Track performance against a separate, held-out data set; stop when performance starts to degrade on this data set (manages over/under fitting but requires some level of smoothing to manage noise)

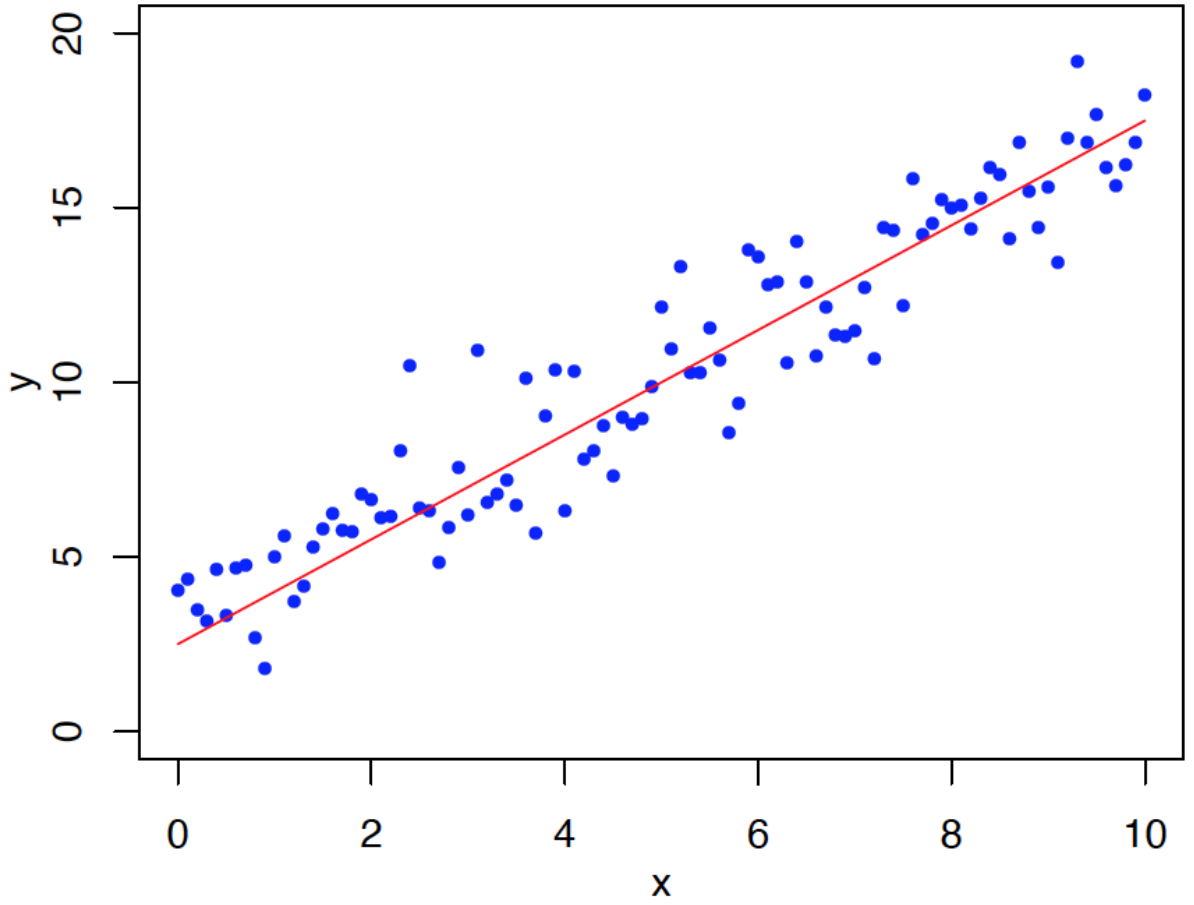
Linear Models

- So far, we have considered learning methods that are not linear:
  - Nonparametric learning such as nearest neighbor
  - Decision trees and rules
- Here we take a step back and consider only linear models. (Recall: Naïve Bayes is linear.)

$$f(\mathbf{x}) = w_0 + \sum_{i=1}^{d} w_i x_i$$

- The objective is to learn the weights $\boldsymbol{\theta} = \mathbf{w} = [w_0, w_1, \ldots, w_d]^\top$.
- We provide this as a foundation for more powerful models to follow.
- We begin by considering the regression problem.

Linear Regression



Linear Regression

- The task is to learn the weights $\mathbf{w}$ in a linear model.
- The typical loss function is mean squared error:

$$\mathcal{L}\big(f(\mathcal{D}, \mathbf{w})\big) = \frac{1}{|\mathcal{D}|}\sum_{i=1}^{|\mathcal{D}|}\Big(y^i - f\big(\mathbf{x}^i, \mathbf{w}\big)\Big)^2$$

- Since the model is linear, this becomes

$$\mathcal{L}\big(f(\mathcal{D}, \mathbf{w})\big) = \frac{1}{|\mathcal{D}|}\sum_{i=1}^{|\mathcal{D}|}\left(y^i - w_0 - \sum_{j=1}^{d} w_j x_j^i\right)^2$$

- This can be solved analytically: $\mathbf{w} = (\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{y}$ where $\mathbf{X}$ is the data matrix (augmented by a vector of 1's for the intercept) and $\mathbf{y}$ is the response vector.

Assumptions

- The error (residual) terms are independent.
- The error (residual) terms all have the same variance across the entire regression function (homoscedastic).
- The errors all have a conditional mean of zero.
- The covariates (i.e., features) are linearly independent.
- Sometimes it is assumed that the errors are normally distributed

Linear Regression by Gradient Descent

- Solving the linear regression problem involves inverting the matrix $\mathbf{X}^\top\mathbf{X}$.
- For a large data set, this can be very expensive.
- Given the assumptions just presented, the loss function is convex, but note that some of the assumptions (e.g., equal variance and Gaussian residuals) can be relaxed.
- Therefore, applying gradient descent will find the globally optimal set of weights, under the added assumption that the data is truly linear.
- Any of the three methods (iterative, batch, or mini-batch) can be used; however, the full batch approach is typically applied for stability.
- Here, $\nabla_{\mathbf{w}}\mathcal{L} = (y_i - (\mathbf{w}^t)^\top\mathbf{x}_i)\,\mathbf{x}_i$.

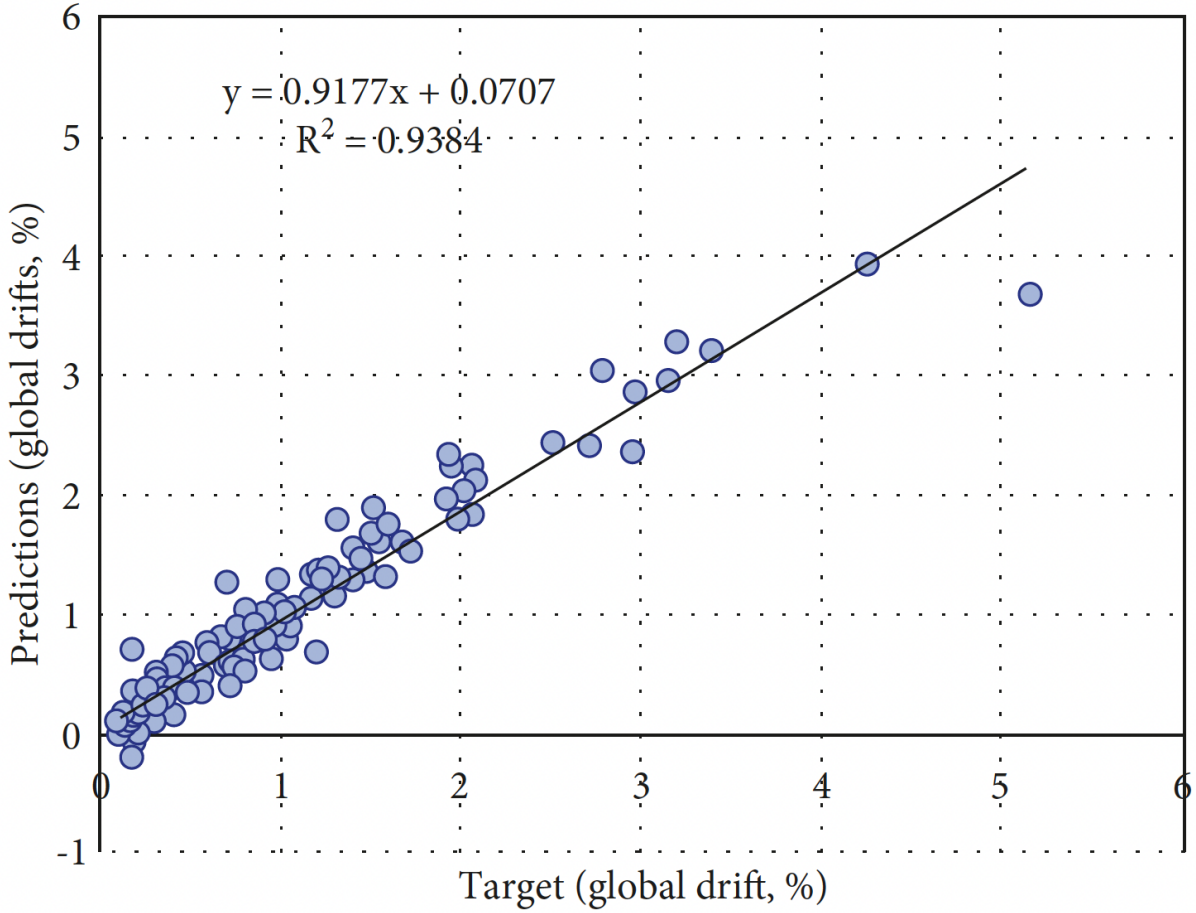
Batch Gradient Descent (Reminder)

- Assume we have a training set, $\mathcal{D} = \{\mathbf{x}_i, r_i\}_{i=1}^{|\mathcal{D}|}$.
- Initialize parameters $\mathbf{w}^0$
- Repeat
  - $t \leftarrow t + 1$
  - $\Delta\mathbf{w} \leftarrow 0$
  - For $i = 1, |\mathcal{D}|$ do
    - $\Delta\mathbf{w} \leftarrow \Delta\mathbf{w} + \eta(y_i - (\mathbf{w}^t)^\top \mathbf{x}_i)\,\mathbf{x}_i$
  - End For
  - $\mathbf{w}^t \leftarrow \mathbf{w}^{t-1} + \frac{1}{|\mathcal{D}|}\Delta\mathbf{w}$
- Until some convergence property is met

Coefficient of Determination

- In addition to considering mean squared error, it is common to evaluate the results of linear regression using the *coefficient of determination* (i.e., R^2).
- Note that R^2 is only valid for *linear* relationships.
- Reflects the proportion of the variation in the dependent (response) variable predictable by the covariates (features).
- Define $e^i = y^i - f^i$ as the "residual" of a prediction.
  - Residual sum of squares: $SS_{res} = \sum_i \left(y^i - f^i\right)^2 = \sum_i \left(e^i\right)^2$
  - Total sum of squares: $SS_{tot} = \sum_i \left(y^i - \bar{y}\right)^2$
  - Coefficient of determination: $R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$

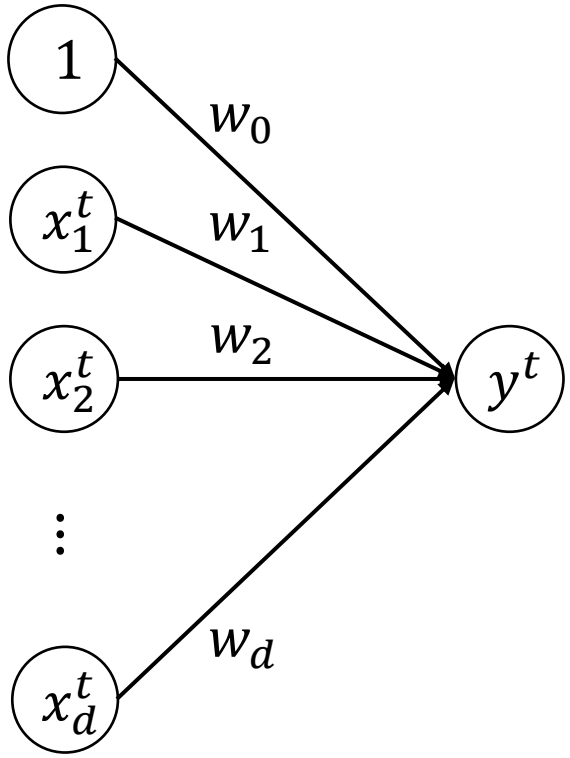
Using R^2



A. Rasheed, M. Usman, M. Zain, and N. Iqbal, "Machine Learning-based Fragility Assessment of Reinforced Concrete Building," *Computational Intelligence and Neuroscience*, Vol. 2022, 5504283.

A Neural Network View

Looking ahead, linear regression can be represented in a network form:



Discriminant Function

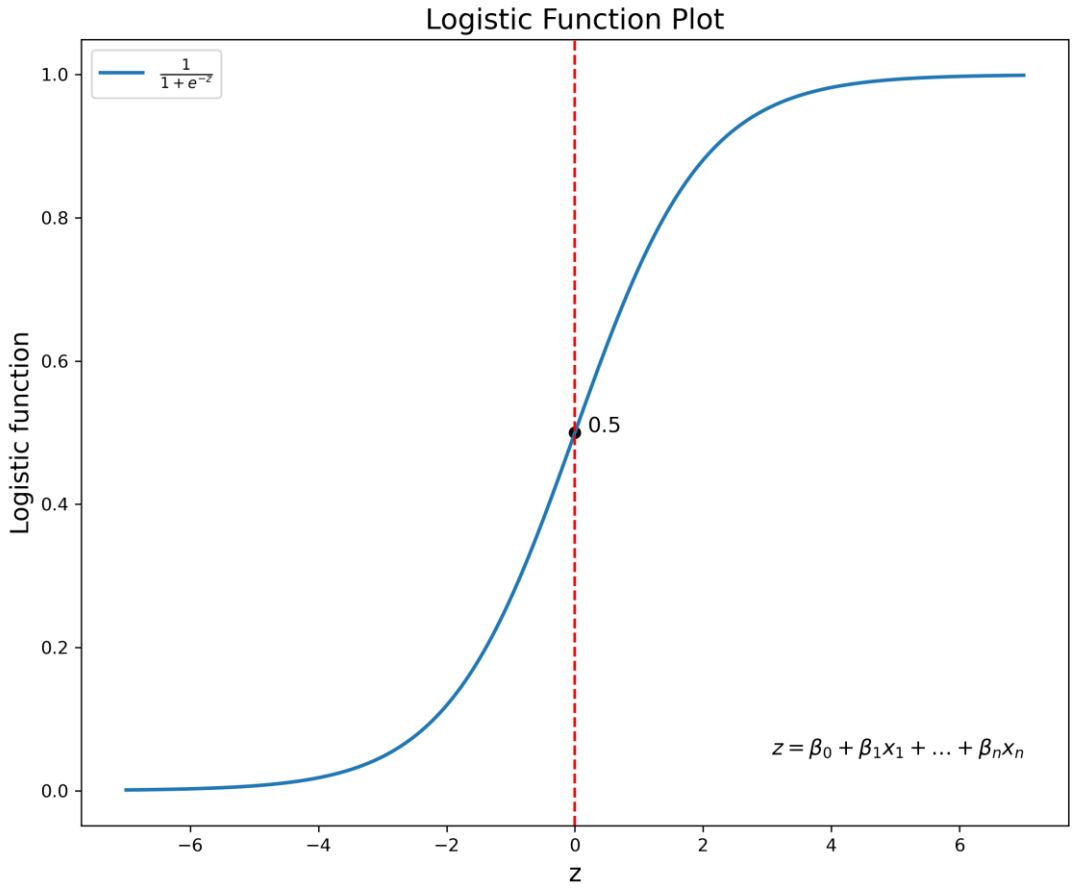
- Maximize posterior probability: $P(c|\mathbf{x})$.
- Consider the two-class classification problem, where $c \in \{0,1\}$.
- We seek to find $P(c = 1|\mathbf{x})$ and $P(c = 0|\mathbf{x})$.
- We use the logit distribution

$$\text{logit}(P) = \log \frac{P}{1 - P}$$

- Suppose we consider the natural log. Then we derive the logistic function:

$$\text{logit}^{-1}(P) = \frac{1}{1 + \exp(-P)} = \frac{\exp(P)}{1 + \exp(P)}$$

Logistic Function



Estimating the Posteriors

- From the logistic function, we get

$$P(c=1|\mathbf{x}) = \frac{\exp\left(w_0 + \sum_{i=1}^{d} w_i x_i\right)}{1 + \exp\left(w_0 + \sum_{i=1}^{d} w_i x_i\right)}$$

$$P(c=0|\mathbf{x}) = \frac{1}{1 + \exp\left(w_0 + \sum_{i=1}^{d} w_i x_i\right)}$$

- Then we choose $c = 1$ if

$$1 < \frac{P(c=1|\mathbf{x})}{P(c=0|\mathbf{x})}$$

$$< \frac{\exp\left(w_0 + \sum_{i=1}^{d} w_i x_i\right)}{\frac{1 + \exp\left(w_0 + \sum_{i=1}^{d} w_i x_i\right)}{1}}$$

Estimating the Posteriors

- Note that this is the same as saying

$$1 < \exp\left(w_0 + \sum_{i=1}^{d} w_i x_i\right)$$

- Taking the log of both sides yields

$$0 < w_0 + \sum_{i=1}^{d} w_i x_i$$

- Notice that this is the linear discriminant function.

Learning the Weights

- The goal is to find weights such that

$$\mathbf{w} = \underset{\mathbf{w}}{\operatorname{argmax}} \prod_k P(C^k | \mathbf{x}^k, \mathbf{w})$$

- It is customary to work in log space since probabilities can be quite small, yielding

$$\mathbf{w} = \underset{\mathbf{w}}{\operatorname{argmax}} \sum_k \ln P(C^k | \mathbf{x}^k, \mathbf{w})$$

- Find conditional log likelihood as

$$\begin{aligned}
\ell(\mathbf{w}) &= \sum_k (C^k \ln P(C^k = 1 | \mathbf{x}^k, \mathbf{w}) + (1 - C^k) \ln P(C^k = 0 | \mathbf{x}^k, \mathbf{w}) \\
&= \sum_k \left(C^k \ln \frac{P(C^k = 1 | \mathbf{x}^k, \mathbf{w})}{P(C^k = 0 | \mathbf{x}^k, \mathbf{w})} + \ln P(C^k = 0 | \mathbf{x}^k, \mathbf{w}) \right) \\
&= \sum_k \left[C^k \left(w_0 + \sum_{j=1}^d w_j x_j^k \right) - \ln \left(1 + \exp \sum_{j=1}^d w_j x_j^k \right) \right].
\end{aligned}$$

Gradient Descent

- The classic approach to solve is via gradient descent.
- We seek $\mathbf{w}$ such that for all w_j

$$\frac{\partial \ell(\mathbf{w})}{\partial w_j} = 0$$

- Note that

$$\frac{\partial \ell(\mathbf{w})}{\partial w_j} = \sum_k x_j^k \left(C^k - \hat{P}\left(C^k = 1 \middle| \mathbf{w} \right) \right)$$

- Weight update rule:

$$w_j \leftarrow w_j + \eta \sum_k x_j^k \left(C^k - \hat{P}\left(C^k = 1 \middle| \mathbf{w} \right) \right)$$

Regularization

- Avoiding learning noise, which would lead to overfitting.
- Penalize large weights.

$$\mathbf{w} = \underset{\mathbf{w}}{\operatorname{argmax}} \sum_{k} \ln P\left(C^k \middle| \mathbf{x}^k, \mathbf{w}\right) - \frac{\lambda}{2} \|\mathbf{w}\|^2$$

- New gradient

$$\frac{\partial \ell(\mathbf{w})}{\partial (w_j)} = \sum_{k} x_j^k \left(C^k - \hat{P}\left(C^k = 1 \middle| \mathbf{w}\right)\right) - \lambda w_j$$

- New update

$$w_j \leftarrow w_j + \eta \sum_{k} x_j^k \left(C^k - \hat{P}\left(C^k = 1 \middle| \mathbf{w}\right)\right) - \eta \lambda w_j$$

Two-Class Logistic Regression

For $j = 0, \ldots, d$
  $w_j \leftarrow$ rand(-0.01,0.01)
Repeat
  For $j = 0, \ldots, d$
    $\Delta w_j \leftarrow 0$
  For $t = 1, \ldots, N$
    $o \leftarrow 0$
    For $j = 0, \ldots, d$
      $o \leftarrow o + w_j x_j^t$
    $y \leftarrow \text{sigmoid}(o)$
    $\Delta w_j \leftarrow \Delta w_j + (r^t - y) x_j^t$
  For $j = 0, \ldots, d$
    $w_j \leftarrow w_j + \eta \Delta w_j$
Until convergence